



Technical Record

Human Activity Recognition on Unseen People (MotionSense)

CW1

Isuru Amarasinghe | KIC-BSCCSAI-251P-042

Kandy Innovation Centre
National Institute of Business Management
1095, Kandy - Colombo Rd, Getambe.

Repository: https://github.com/isuruamarasingheu/motionsense-har-unseen-subjects
Word count = 1,490

T1 Problem definition and data preparation

a. Problem and provenance. A model to predict whether someone is going downstairs, upstairs, walking, jogging, standing or sitting based on 2.56 seconds of smartphone movement data (128 samples, 50Hz sampling rate, 12 sensors). The aim is to see if the model generalizes well to unseen individuals. The data comes from the MotionSense dataset (Malekzadeh et al, 2019). It is available under the MIT license from its GitHub Repo [github.com/mmalekzadeh/motion-sense]. There are 24 individuals in the cleaned dataset, 359 recordings, 1,402,573 rows (7.79 hours) and 21,380 windows. The ethical implications include that individuals can be identified based on their gait when walking and age, gender, height and weight are stored in the dataset.

b. Data audit. No missing data. Subject 1 had two data sets that were identical (sit_5 and sit_13, 10,292 rows). One was dropped (sit_13). There is an imbalance between classes. The training contains 3488 windows of sit, but only 1226 windows of down (approx. 2.8:1, see Fig. 1). The channels have very different ranges (see Fig. 4), all features are normalized. The data appears similar for walk and stair (Fig. 2).

c. Leakage-safe pipeline. If the data were randomly split the same person would be present in both training and testing, allowing the model to identify individuals rather than activities, giving a falsely positive result (identity leakage). Therefore, the data is split by person (random seed 42). 16 subjects in training (subjects 1, 8, 18, 21, 2 males and 2 females), 4 in validation and 4 in testing. No windows overlap between recordings. Scalers are only fit on training data. No data augmentation. Only features from the window are used. Person or trial number are never given to the model (no target leakage). Data is split and scalers stored. Testing ensures no person is in more than one split.

T2 Learning from first principles (NumPy)

a. Forward pass. Fully connected neural network with 84 input nodes, 32 hidden units, and 6 outputs. H = tanh(XW1 + b1), P = softmax(HW2 + b2). Loss function = mean cross entropy. X (B, 84), W1 (84, 32), b1 (32), W2 (32,6), b2 (6), P (B,6). Each hidden unit performs a calculation using weights on the inputs and a non-linear curve like a brain cell. However, there are no spikes, and they learn by back propagating their errors.

b. Backpropagation. $dZ2 = (P - Y)/B, dW2 = H^T\ dZ2, dZ1 = dZ2\ W2^T \odot (1 - H^2), dW1 = X^T\ dZ1$. Mini-batch gradient descent with learning rate 0.05, batch size 64, Xavier initial weights. Trained with 3000 training windows. Loss decreased from an initial value of 1.877 (close to ln(6)=1.792 which is what you would get by guessing randomly) down to 0.112 (training) and 0.198 (validation). Macro-F1 score on validation was 0.925 (Figure 5).

c. Gradient check. Gradients were checked numerically using central difference scheme with ε=1e-5. Gradients were calculated for 8 different values in W1, b1, W2 and b2 for a batch of 8. The maximum relative error was 1.2e-10 which is well within our acceptable error of 1e-6. Since tanh is smooth there are no kinks.

d. Numerical diagnosis. The training was stopped after 15 epochs when the validation loss had not improved. The best epoch was 25 and the model was trained for 40 epochs. The model was able to learn the small batch of 32 training windows perfectly. The loss decreased from 1.949 to 0.0036 with 100% accuracy after 2000 epochs (Figure 6). This shows that the updates are working. The learning rate was tested and it was found that there was an issue with the hidden layer. With a learning rate of 2.0 the training loss decreased to 0.0004 but the validation loss increased to 0.360 (from 0.218). The macro F1 score remained around 0.92. With higher learning rates the model became over-confident.

T3 Baseline and framework models

a. Baseline. A Random Forest (RF) on the same 84 features and splits, scored the same way.

b. Two neural networks (layer tables in Appendix A):

- **MLP on features**: 84 → 128 → 64 → 6, with BatchNorm, ReLU and dropout 0.2 (19,910 parameters). It cannot see the time order of readings.
- **1D-CNN on raw signals**: 12 channels, 128 timesteps. 3 conv blocks with 64, 128 and 128 filters (size 5), followed by BatchNorm, ReLU and Pooling. Global averaging, dropout of 0.3 and an output layer. Total parameters = 128,454. Learns temporal patterns directly.

The two have different architectures (hand-crafted features and learned filters). Both predict 6 scores, using cross-entropy loss. Initial weights of MLP were set to Kaiming normal (as ReLU is used). Initial weights of CNN were set to default Kaiming uniform in PyTorch. Adam was used as the optimiser (lr=1e-3, weight decay=1e-4)

c. Training control. Batch size of 64, max 60 epochs. Weight decay and dropout to reduce overfitting. Training stopped early if no improvement in validation macro F1 score for 8 epochs. Best weights were also saved. Plots shown in figs 8-9.

d. Traceability and fair tuning. Data processing, training and evaluation were done in separate files. Every run saved the configuration, model and details. Python version = 3.11.9, Torch version = 2.15.0+cpu, Scikit-learn version = 1.7.2, Processor = Intel i5-11400H, OS = Windows. Budget for tuning was specified in advance (tuning_budget.yaml). 6 trials per model, seed=0. The best model was selected based on validation macro F1 score (Appendix B). No code or pre-trained models were used. Unequal computation times. CNN took between 26-132 seconds per trial whilst MLP took between 6-13 seconds. The CNN also takes raw signals, so no features.

T4 Experiments and evaluation

a. Repeated evaluation. Final settings were frozen, each model was trained with seeds 0, 1 and 2, and the test set was run once. Plots use the seed with the middle validation score, never the best test score.

Table 1. Results (macro-F1; test = unseen subjects 1, 8, 18, 21).

Model	Config	Seed	Best epoch	Val macro-F1	Test macro-F1	Test acc.
RF	baseline_rf_final.yaml	0 / 1 / 2	–	0.937 / 0.928 / 0.926	0.886 / 0.888 / 0.884	0.881 / 0.884 / 0.881
MLP	mlp_final.yaml	0 / 1 / 2	11 / 15 / 17	0.956 / 0.953 / 0.963	0.882 / 0.894 / 0.879	0.888 / 0.898 / 0.879
CNN	cnn_final.yaml	0 / 1 / 2	18 / 3 / 4	0.967 / 0.944 / 0.946	0.842 / 0.895 / 0.847	0.843 / 0.902 / 0.851
RF mean ± std				0.930 ± 0.006	**0.886 ± 0.002**	0.882 ± 0.002
MLP mean ± std				0.957 ± 0.005	**0.885 ± 0.008**	0.888 ± 0.009
CNN mean ± std				0.953 ± 0.013	**0.861 ± 0.029**	0.865 ± 0.032

The RF and MLP models show similar performance. The CNN seed 0 has the highest validation accuracy (0.967) but lowest testing accuracy (0.842). This cannot be relied upon due to there only being four people within the validation set. All models have a drop in performance of between 0.04 and 0.09 (Fig 14)

b. Metrics. The primary metric used is macro-F1 due to giving equal weighting to all activities including less frequent activities. Accuracy, precision, recall and F1 were also recorded for each class (Table 2, Fig 13). The most common error made was predicting walking as upstairs (RF 214, MLP 250, CNN 340 windows). This reduces precision for upstairs and recall for walking. The RF also has some confusion between sitting and standing (111 windows).

Table 2. Per-class test precision / recall (representative seeds: RF 1, MLP 0, CNN 2).

Class	RF P / R	MLP P / R	CNN P / R
dws	0.91 / 0.89	0.91 / 0.86	0.92 / 0.69
ups	0.65 / 0.96	0.60 / 0.93	0.50 / 0.91

Class	RF P / R	MLP P / R	CNN P / R
wlk	0.94 / 0.73	0.92 / 0.65	0.87 / 0.59
jog	1.00 / 0.96	0.91 / 0.94	0.99 / 0.97
std	0.87 / 1.00	0.98 / 1.00	0.96 / 1.00
sit	1.00 / 0.86	1.00 / 1.00	1.00 / 0.98

c. Ablation. Adding dropout of 0.3 before the CNN output increased the validation set macro-F1. Only the dropout rate was altered (set to 0.0). This resulted in a macro-F1 of 0.9525 ± 0.013 and 0.9513 ± 0.021 for the validation set (a 0.001 increase). This is much less than the variance between seeds (Fig. 15). Therefore, the claim is not supported. Early stopping was used meaning that the number of epochs varied greatly between 3 and 18. Only 3 seeds were used meaning that it was difficult to know if the model was stable.

d. Errors and robustness. 6 common errors were analyzed (Fig. 16). The errors mainly occur for a few recordings. For example, during subject 1's walking trial 7, the CNN made 81 errors out of 83 windows. The median pitch of the phone was -0.86 rad compared to -1.28 rad for other walks by the same person. This means that the phone was angled. However, this does not prove that this was the cause.

Stress Test - Noise (Pre-defined method: noise_protocol.json). Random noise was added to the test signals. There were 5 levels of σ (σ = 0-1.0 times the range of the training set for each channel). This meant there were 5 samples, 3 seeds and 3 models. This resulted in 3730 windows. It was expected that the CNN would perform worst. However, at σ = 0.5, the CNN still performed well with a macro-F1 of 0.606 ± 0.011. The MLP and RF both performed badly with macro-F1 scores of 0.217 and 0.233 respectively (Fig. 17). The MLP performed better for females (0.946) than males (0.819). However, there were only two people in each category meaning this is due to individual differences.

T5 Recommendation and use

a. Recommendation. Speed: i5-11400H CPU, single window including pre-processing. Average time over 100 runs (after 10 warm-up runs). RF uses 1 thread, PyTorch uses 6. The MLP would be recommended. It has a similar score to the RF (0.885±0.008 vs 0.886±0.002) but is much smaller (280x smaller - 0.08MB vs 22.37MB) and faster (16x faster - 0.48ms vs 7.56ms). The RF is simpler so best if consistency is most important (lowest variability between seeds). The CNN isn't really a good choice unless the sensors are noisy (it performs worse with clean data) and is the least stable (Fig 19).

b. Inference pipeline. `python -m src.predict --model {rf,mlp,cnn} --input file.csv` loads the saved model and scaler. It outputs the activity and average confidence. A normal recording from an unseen subject (subject 8, walking) will output WALKING (0.95). If the file is too short (60 rows), missing columns or contains empty values then it will return an exit code of 1 and an informative message. If the file has some unusual ranges then there will just be a warning and it will classify it (JOGGING, 1.00).

c. Limitations. Bias: Only 24 subjects (aged 18-46), one phone model, one position and just 4 test subjects. Generalisation: The scores drop when tested on new subjects and tilted phones. None of the models are ready for real-world use. Privacy and abuse: Movement data can identify who someone is, their health and habits. Predictions should therefore be done on the phone with permission. In the next experiment the model will be trained on randomly rotated signals.

T6 Reproducibility and demonstration

The README explains the setup. There are pinned requirements (requirements.txt). Explains how to download the data (can't be redistributed). Which order to run them. Smoke test (run pytest -q, run CNN for 1 epoch (around 35s)). Split, seeds, final config, model files and results. mlp_final.yaml → pytorch_mlp_seed0.pt → row in results_table.csv → Fig 11. Explains why the data was split by subject and shows how it failed on tilted phones.

Appendix A — Layer tables

PyTorch MLP (input 84 features).

Layer	Output shape	Activation	Params
Linear 84→128 + BatchNorm	(B, 128)	ReLU, Dropout 0.2	10,880 + 256
Linear 128→64 + BatchNorm	(B, 64)	ReLU, Dropout 0.2	8,256 + 128
Linear 64→6	(B, 6)	logits (softmax in loss)	390
Total			**19,910**

PyTorch 1D-CNN (input 12 × 128).

Layer	Output shape	Activation	Params
Conv1d 12→64, k5 + BatchNorm, MaxPool 2	(B, 64, 64)	ReLU	3,904 + 128
Conv1d 64→128, k5 + BatchNorm, MaxPool 2	(B, 128, 32)	ReLU	41,088 + 256
Conv1d 128→128, k5 + BatchNorm, MaxPool 2	(B, 128, 16)	ReLU	82,048 + 256
Global average pool + Dropout 0.3	(B, 128)	–	0
Linear 128→6	(B, 6)	logits	774
Total			**128,454**

Appendix B — Tuning trials (validation macro-F1, seed 0; selected in bold)

Model	Trial settings → val macro-F1
RF	100 trees: depth None 0.927, 20 0.929, 10 0.905; 300 trees: **None 0.937**, 20 0.937, 10 0.909 (all balanced)
MLP	[128,64] d0.2 **0.956**; [128,64] d0.4 0.953; [256,128] d0.2 0.944; [256,128] d0.4 0.945; [128,64] d0.2 weighted 0.947; [256,128] d0.2 weighted 0.934
CNN	[32,64,64] k5 d0.3 0.938; k7 d0.3 0.940; k5 d0.5 0.955; [64,128,128] **k5 d0.3 0.967**; k7 d0.3 0.950; k5 d0.5 0.964

Appendix C — Figures

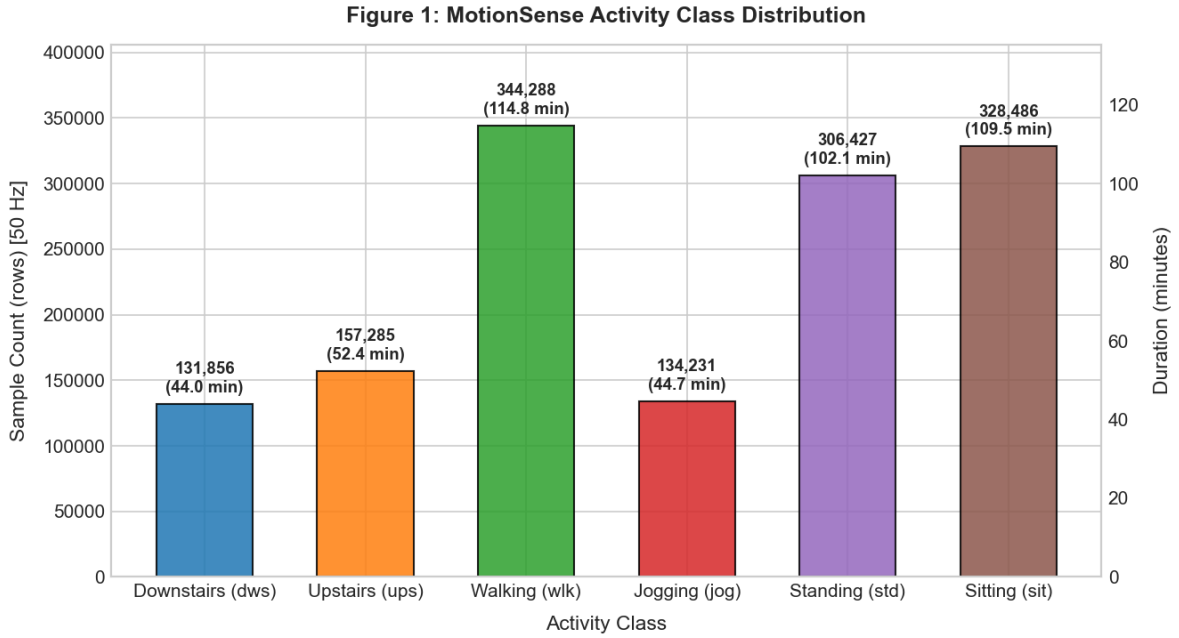


Figure 1. Class distribution: data per activity (T1b).

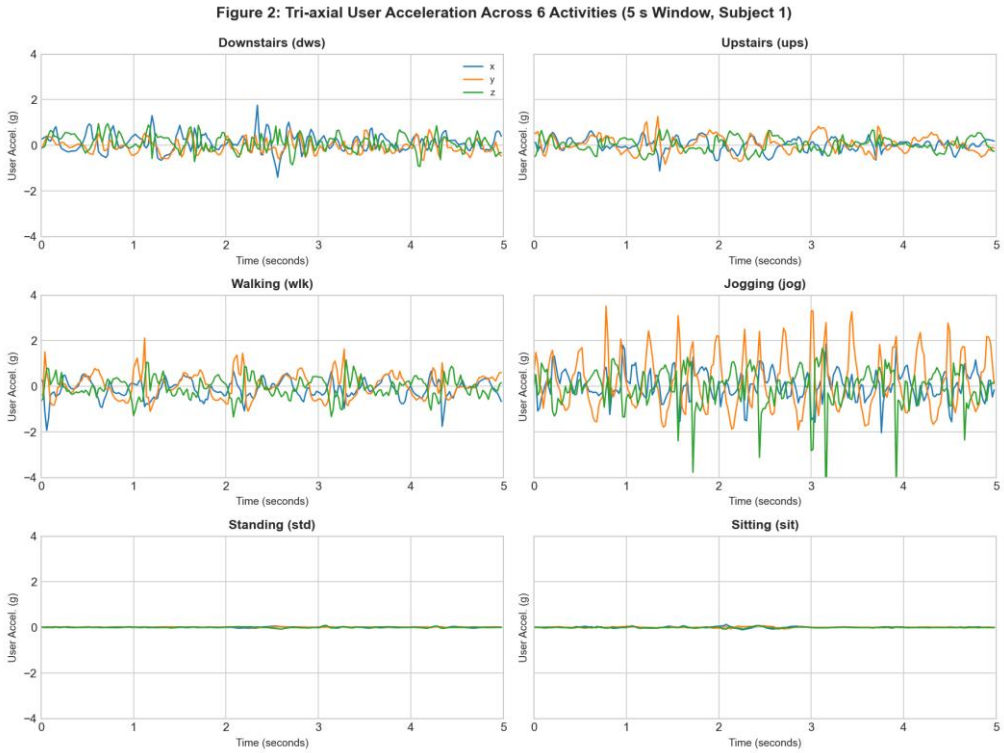


Figure 2. Example user-acceleration signals for each activity (T1b).

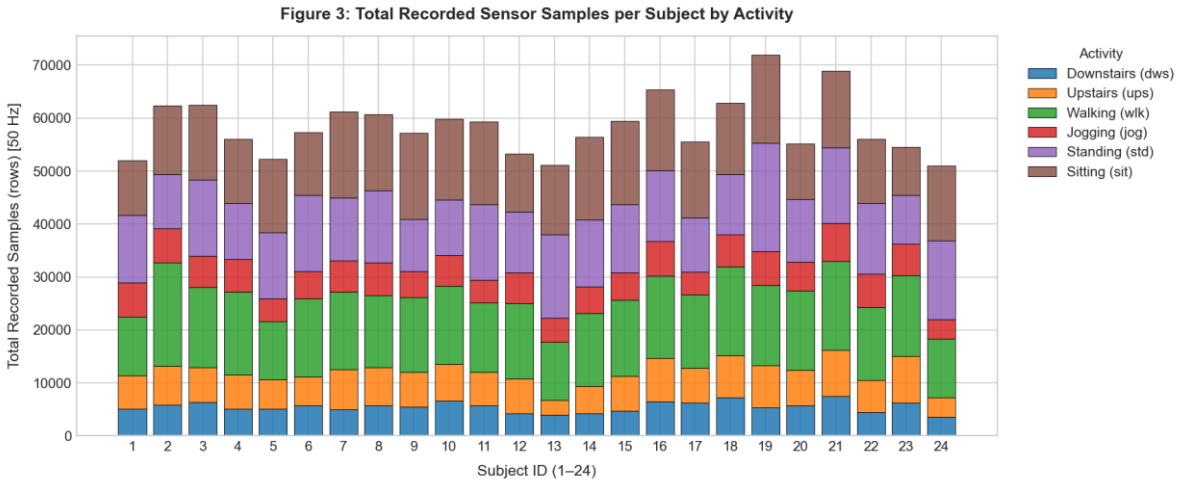


Figure 3. Amount of data per person, split by activity (T1b).

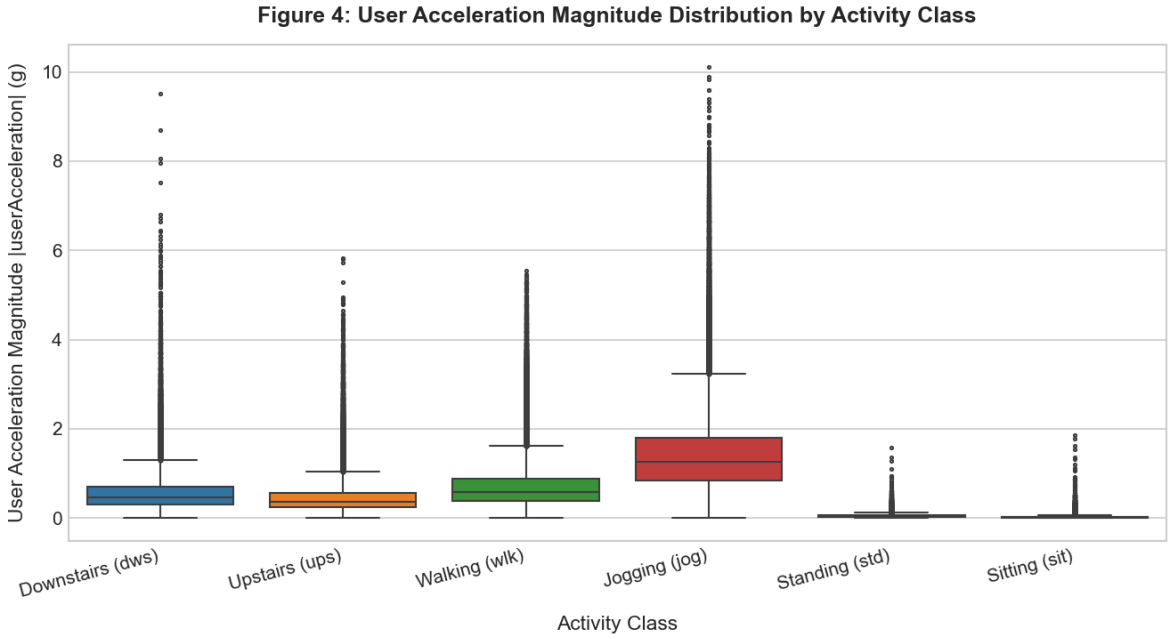


Figure 4. Movement strength (acceleration magnitude) per activity (T1b).

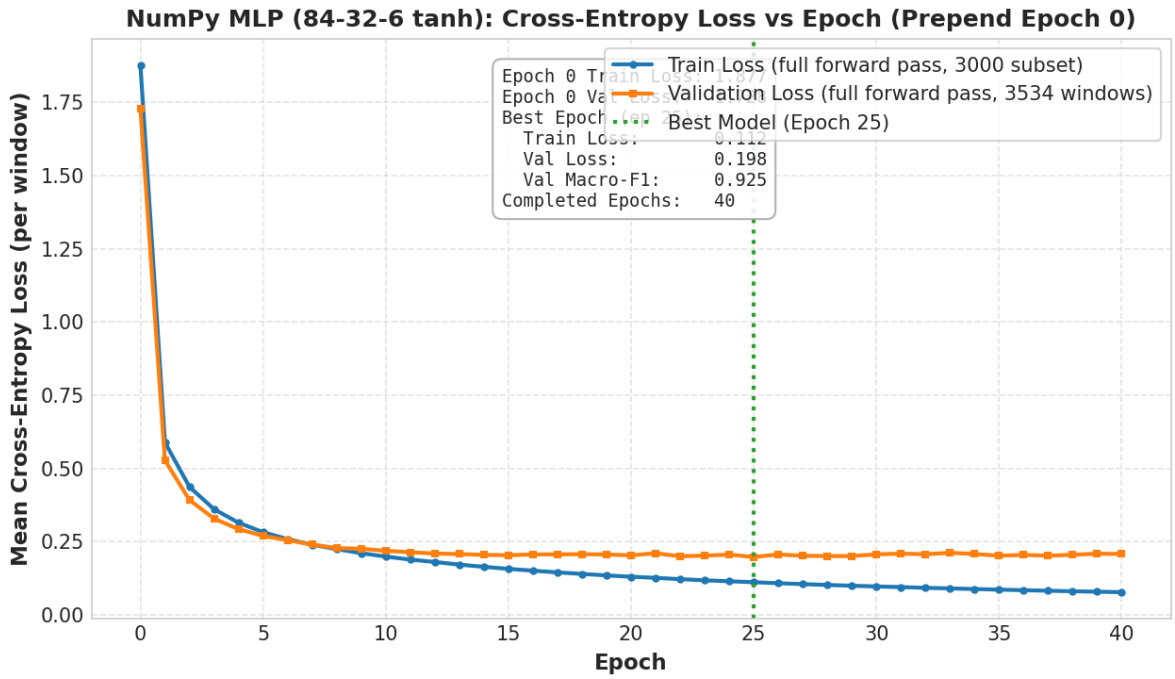


Figure 5. NumPy MLP training and validation loss with early stopping (T2b).

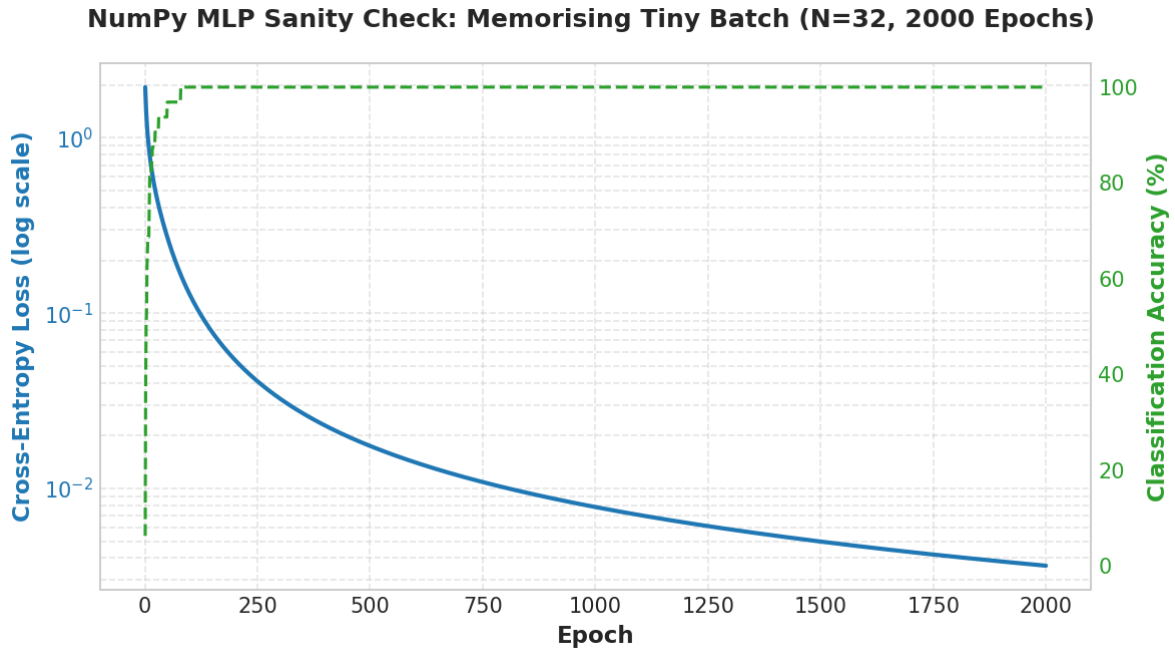


Figure 6. NumPy MLP memorising a tiny batch of 32 windows (T2d).

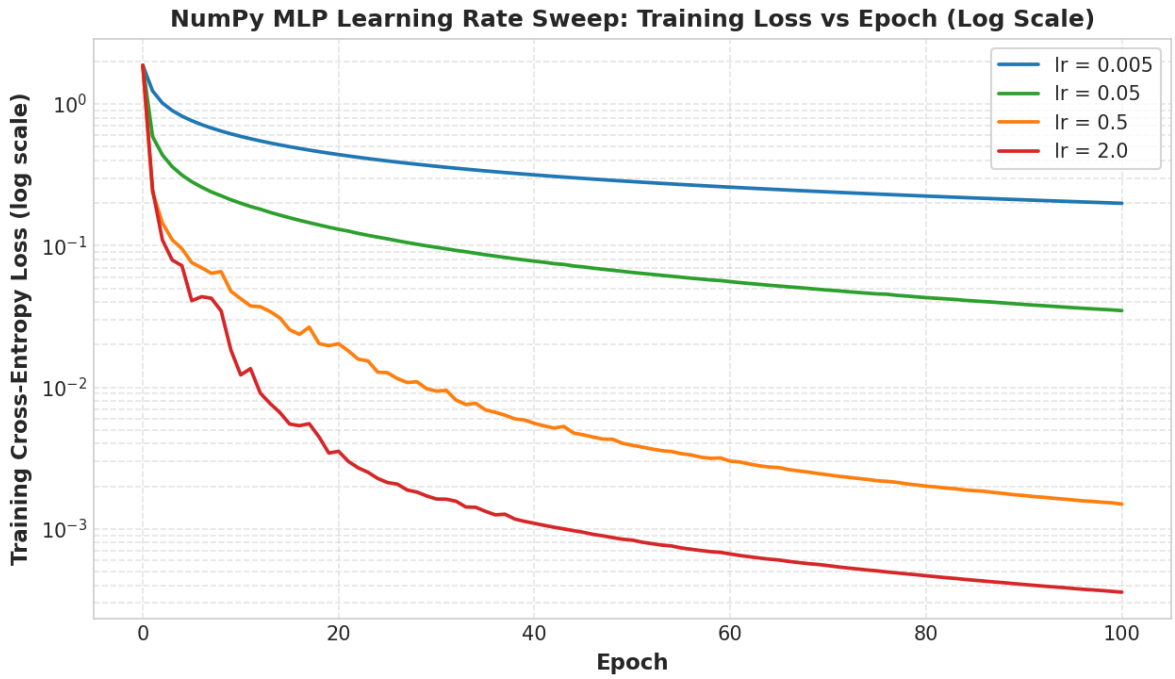


Figure 7. NumPy MLP learning-rate sweep (T2d).

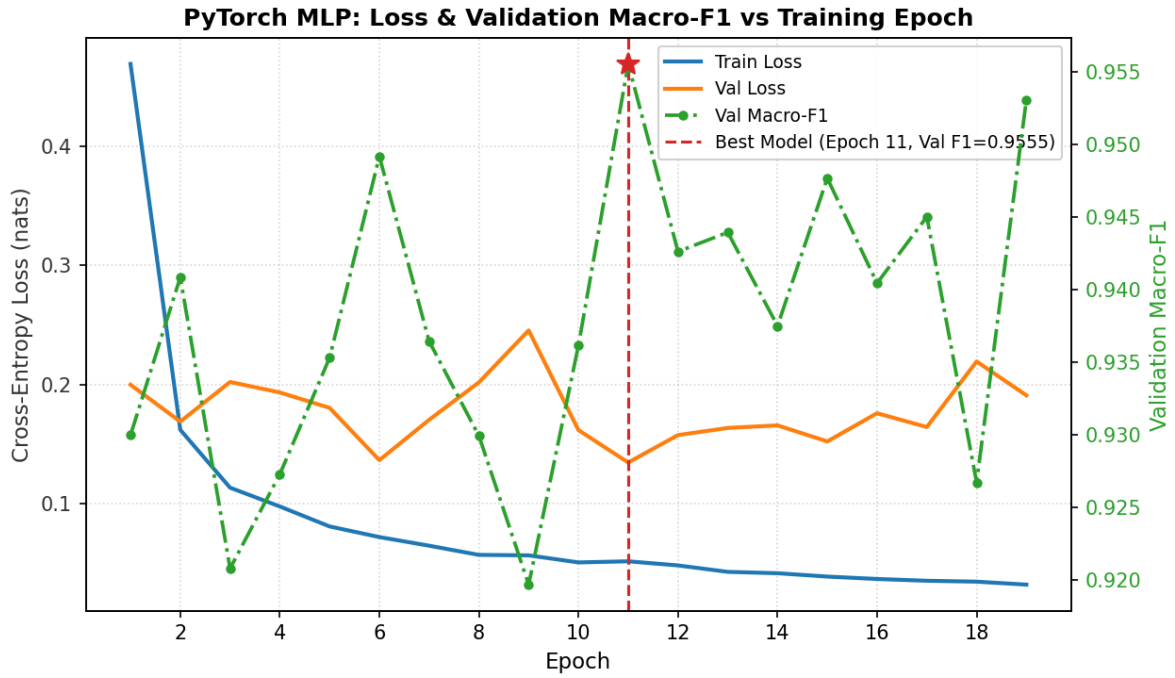


Figure 8. PyTorch MLP training/validation loss and validation macro-F1 (T3c).

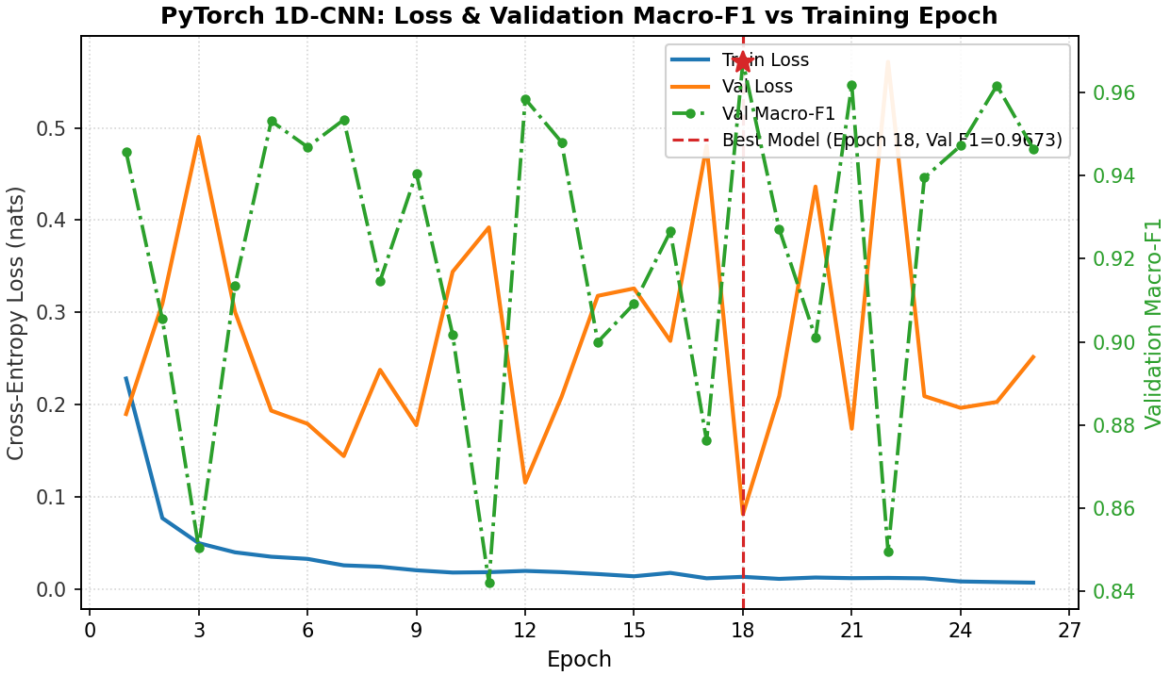


Figure 9. PyTorch 1D-CNN training/validation loss and validation macro-F1 (T3c).

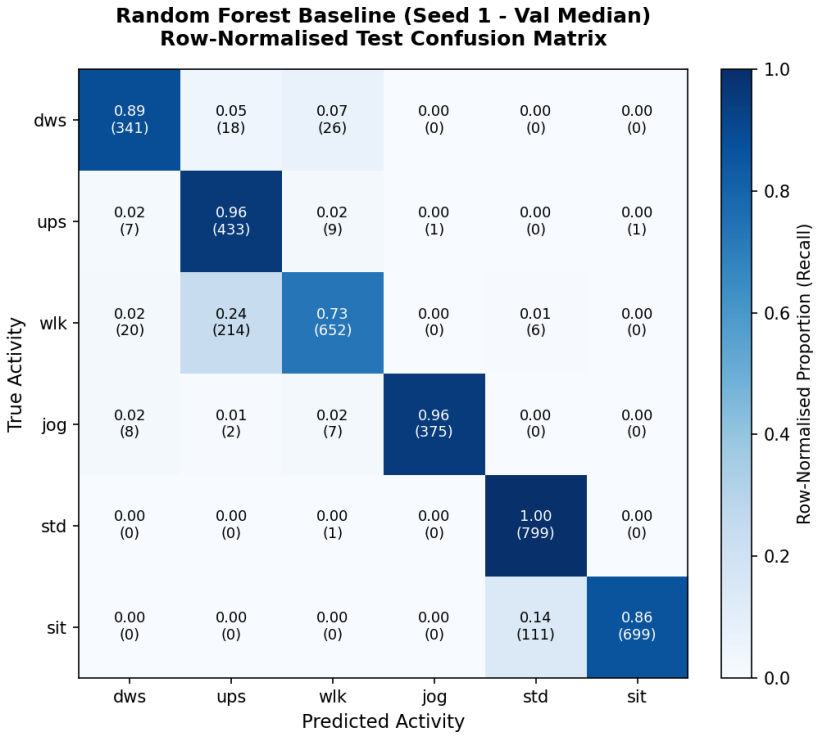


Figure 10. Random Forest test confusion matrix, seed 1 (T4b).

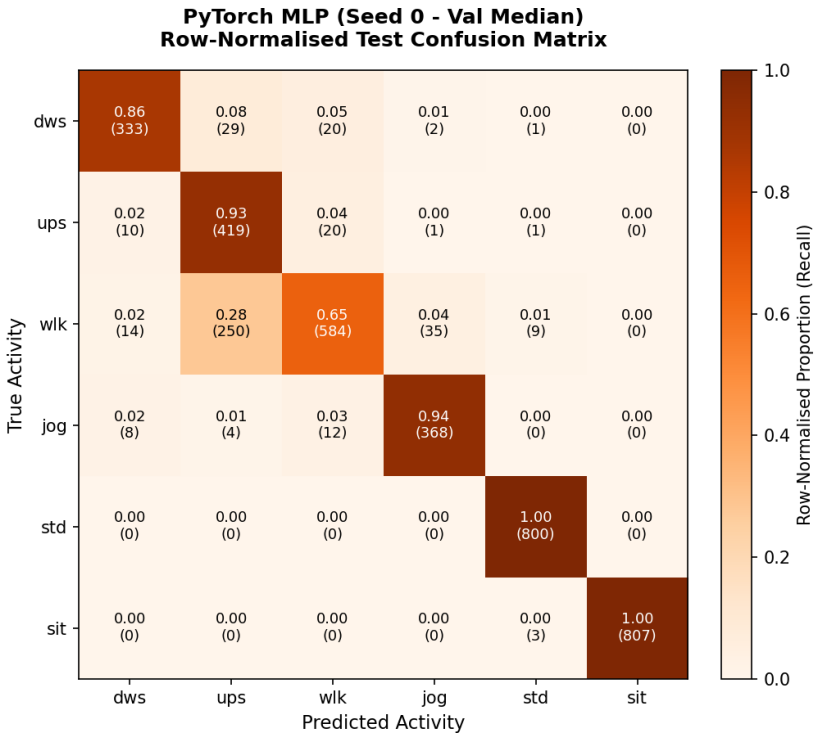


Figure 11. PyTorch MLP test confusion matrix, seed 0 (T4b).

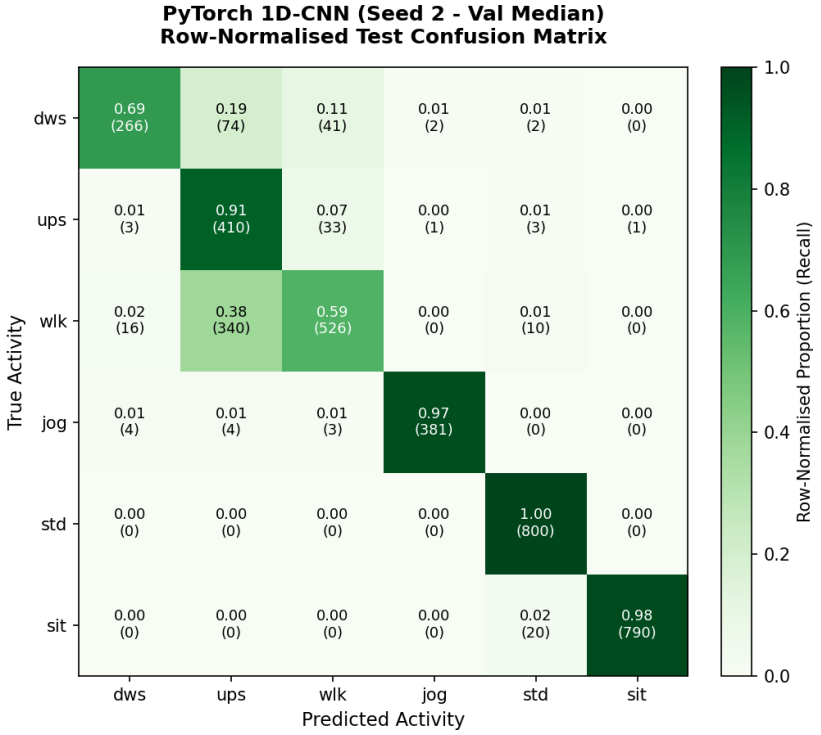


Figure 12. PyTorch 1D-CNN test confusion matrix, seed 2 (T4b).

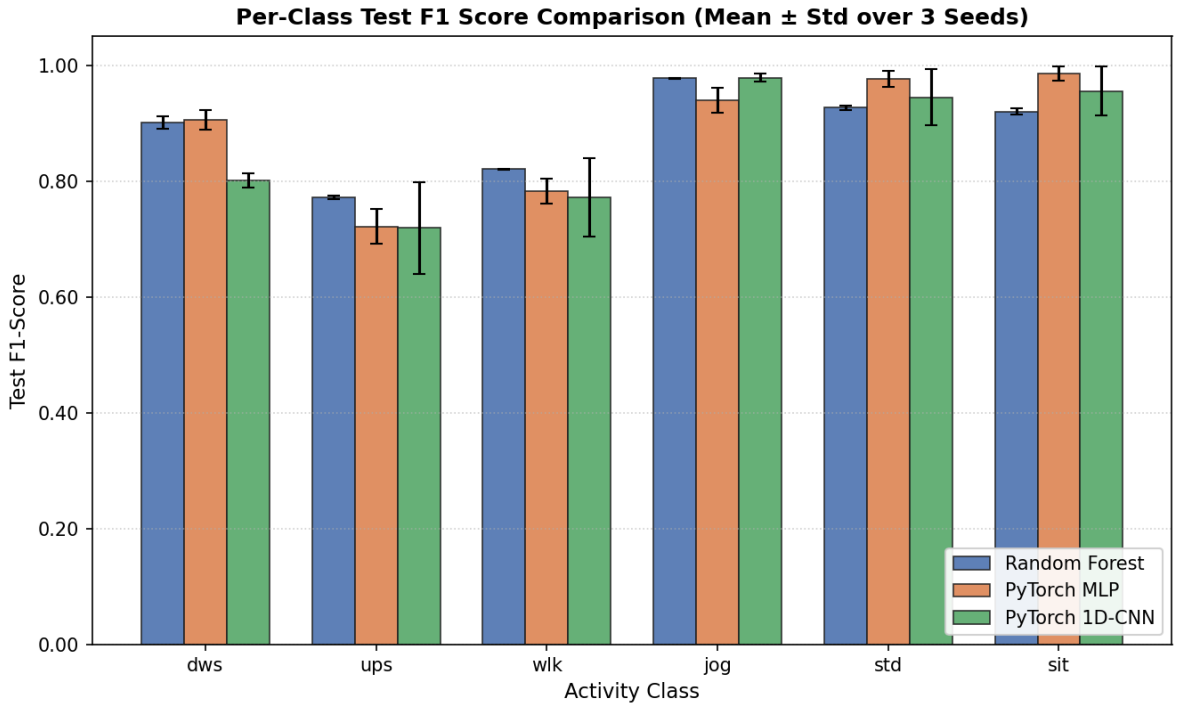


Figure 13. Per-class test F1, mean ± std over 3 seeds (T4b).

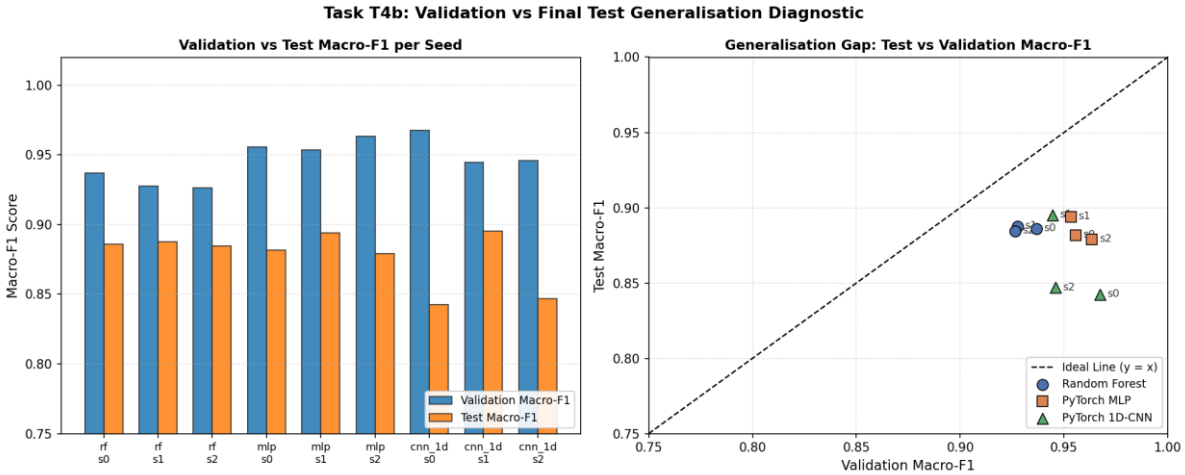


Figure 14. Validation vs test macro-F1 for each model (T4a).

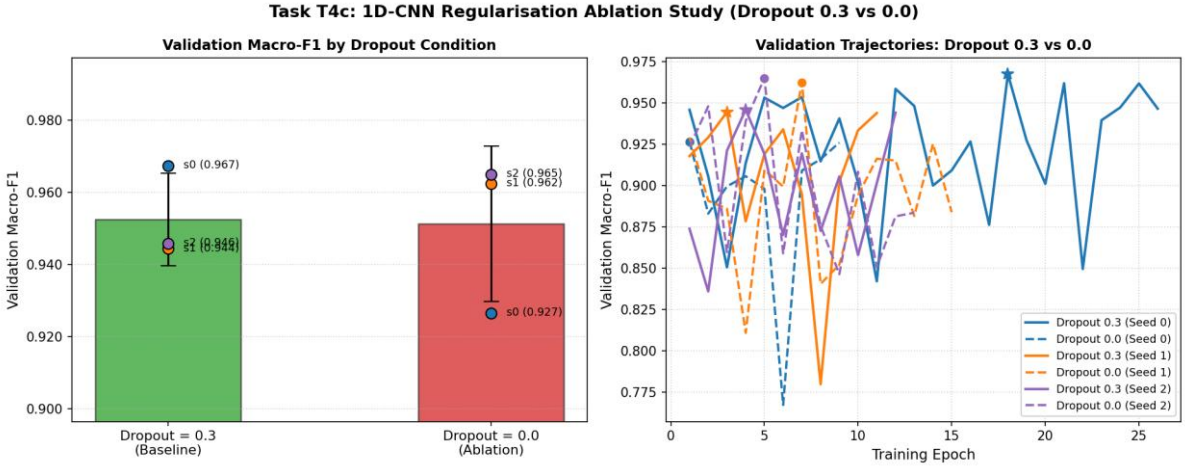


Figure 15. 1D-CNN dropout ablation on validation data (T4c).

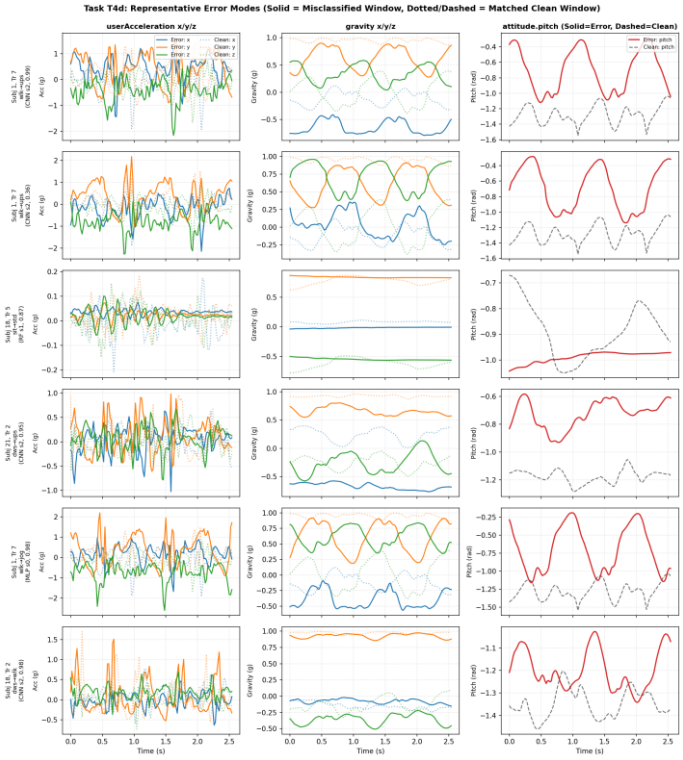


Figure 16. Sensor signals for representative errors (T4d).

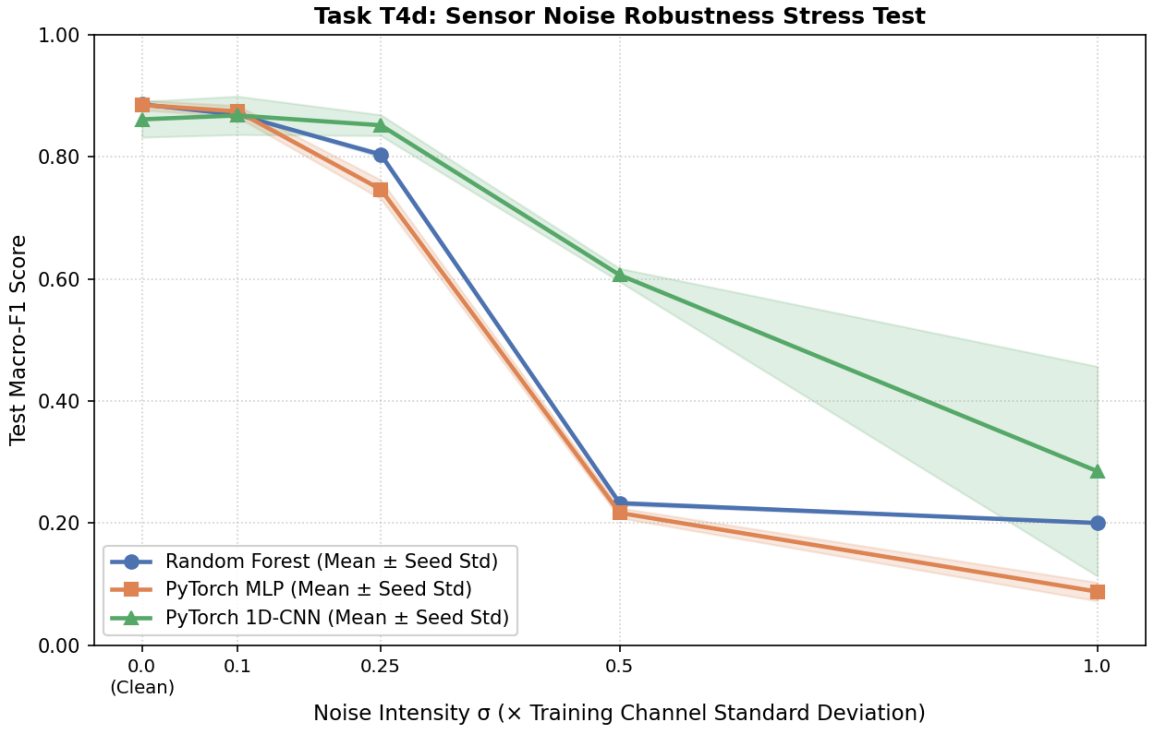


Figure 17. Test macro-F1 under increasing sensor noise (T4d).

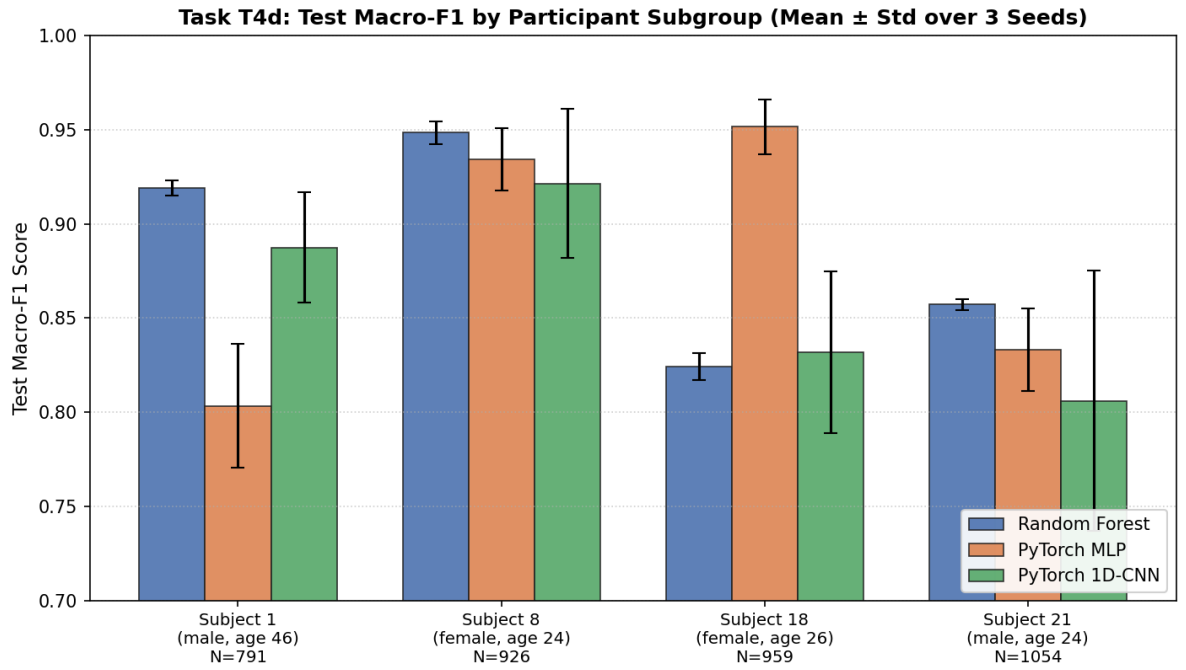


Figure 18. Test macro-F1 for each of the 4 test people (T4d).

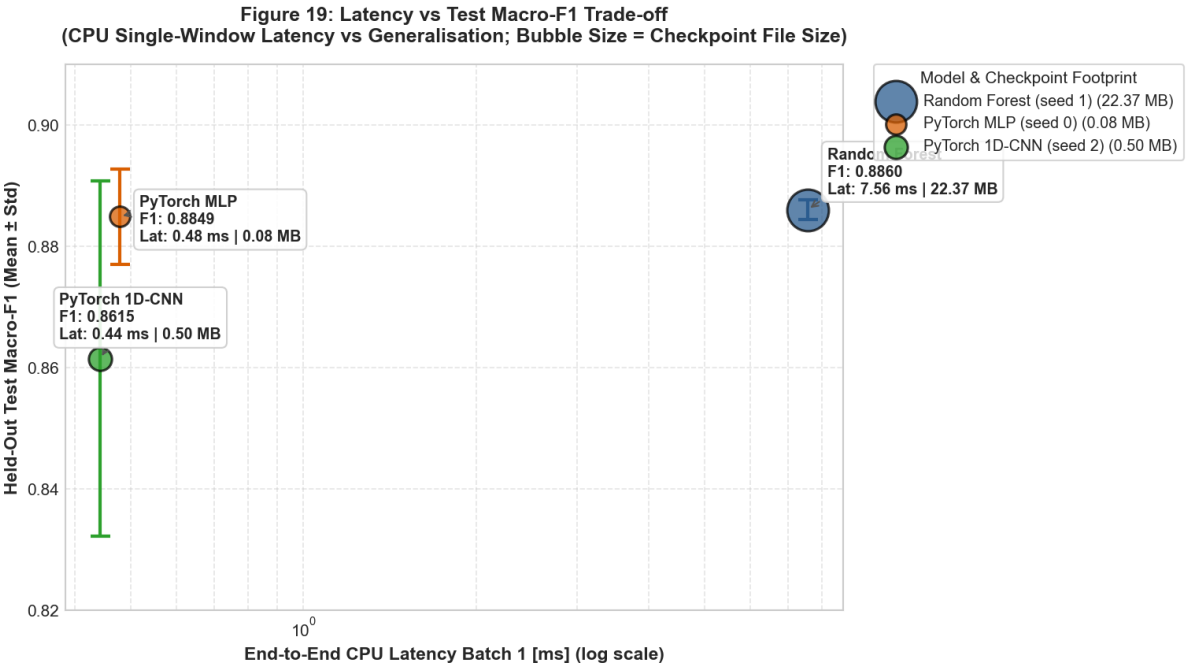


Figure 19. Inference latency vs test macro-F1; bubble size = model file size (T5a).

References

Malekzadeh, M., Clegg, R. G., Cavallaro, A. and Haddadi, H. (2019) 'Mobile sensor data anonymization', *Proceedings of the International Conference on Internet of Things Design and Implementation (IoTDI '19)*. ACM. Dataset: https://github.com/mmalekzadeh/motion-sense